

Project Report

ClassLLM - Topics Generator Feature

Created By Claude-GPT-Mini: Idan Harel, Rom Waidergoren, Ilay
Amsalem, Shon Grinberg



This document covers our second feature work in the project ClassLLM : Topics Generator.

1 Overview

ClassLLM is a project developed during the "Software Engineering with LLM" course as part of our Computer Science degree. In this project we managed to create an online chatbot that will be accessible to both students and teachers and would be able to help students learn some topics of their courses, sharpen their understanding and test their knowledge without the need of human staff or additional lecturing hours. Our team's first task in this project was to create the authentication service that allows users to login the system as teacher or student and authorize actions based this role. Another task we had to create a topics generator service that can label each markdown material document containing parts of the course materials with the relevant topics this content related to and generate a visual topics tree that represents the topics hierarchy.

2 Place in the Workflow

After initializing a course there is a need to upload the contents that the chatbot would be based on. The files are being uploaded to the files database and are being passed to our feature for topics creation. The feature extract the topics from the files and updates a topics hierarchy tree that the labeling of the topics that is related to each file will be taken from this tree. After the topics extraction, the topics are being stored with the files ready to be retrieved by the model.

3 Feature Requirements

The feature should have 2 functionalities:

- **Topics Tree Generator:** This functionality should get several "initialization files" that are the core files of the course that could contain course contents, topics, explanations etc., and generate a full course topics tree that is ordered by the topics hierarchy (for example the topic "Bubble Sort" would be a child topic of "Sort Algorithms"). This tree should be visible to check for correctness.
- **File's Topics Extraction:** This functionality should get a file as an input and return the topics (from the topics tree) that relate to the contents in the file. If a topic is found in the file and it is not in the topic tree - it will be added to the tree.

4 Implementation Details

In order to reach the requirements we finally implemented 3 functions:

1. **Topics_Tree(files):** This functions gets files and create topics tree according to their content. Firstly the files are being opened and the text is sent to DSPy prompt generator. the prompt generator ask the LLM to generate the topics on the format of the types. we store the topics as objects and ask the LLM to generate the tree from the objects we created. The tree can be visualized by an HTML.
2. **Extract_Topics(file):** This function gets a file and generate the topics labels of its. The functions uses DSPy to create a prompt that includes the text of the file, the topics of the current tree and the format we want the topics to be stored on. the LLM gets the prompt and generate the list of topics the file contains and store them as objects. If there is another topic that is in the file but not in the tree yet, it triggers the Update_Topics_Tree function to add it to the tree.
3. **Update_Topics_Tree(topic):** This function gets a new topic objects and adds it to the tree. The function uses DSPy to create a prompt that takes the new topic and the current tree and add it to the right place according to the hierarchy of the topic. The returned tree is the new topics tree.

5 Work Stages

After receiving the task we firstly wanted to make an experiment:

We generated a text file containing some course materials and reformatted the text into a prompt that was given as an input to an LLM and tried to get the topics in return. The result were a mixed when we got a lot of topics (even some that are not taking a serious part in the material that was mentioned as a notation), some of the topics were duplicates or similar (like "recursion" and "recursive functions"). From this experiment we drew two conclusions:

- **Upgrade the Prompt:** We understood that in order to get the topics in the right format and as we need to use external tools to create the best prompt for the task so we decided to use DSPy to create a much better prompt.

- **Union Similar Topics:** For merging similar topics (like "BST" and "Binary Search Tree") we needed to connect each generated topic to a closest topic we already have and recheck the topics to find duplicates and merge them.

So we created a function called `Extract_Topics()` that uses DSPy to create the prompt that contained the explanation of our need, the format we wanted to get the output in, and the overview of the topics to find duplicates and merge them. the first tries of this was much better than previous tries - we got a list of topics (even though there were many topics) that didn't contain duplicates and was quite accurate. Now we faced another challenge - different executions of the functions on different files can create similar (but different) topics between files that related to the same topic - a problem that can make future connection between them a lot harder. In addition we got an extension to our task to add a tree of topics hierarchy of the course topics. we decided to change the architecture as follows:

In the beginning, the teacher will upload "initialization files" that contain a lot of materials, course content and so on, that will become the base for the topics tree. this file will be processed, their topics will be extracted and stored in the tree. After the initialization of the tree, each file that will be added (including the initialization files) will use the topics from the tree in the `Extract_topics` function and will choose the topics from the tree that related to its contents. new topics that were found in the files, but were not in the tree were added by the `Update_Tree()` function.

This implementation improved the results a lot and created the topics tree as we wanted. Later on we added a visualization of the tree.

In order to make the tree represent metadata of the topics we created a new type of topic that contains: name, description, metadata (source file, confidence_score, source_section) for storing metadata and Topics Tree type that stores the hierarchies between the topics. This was the final stage that made our visualization be a lot more informative.

6 Results and Conclusions

In the end we got a working topics generator that gets files and can label topics in them and create a hierarchy of the topics for future features usage.

Conclusions:

- **LLM usage:** We can use LLM almost for every task within the project: from writing documentation to implementing and testing it. In this project we started implementing things by ourselves but later on we found out the the most efficient way is to give as much attention to the prompts we need (also use an external tools like DSPy) and then let the LLM help us generate most of the implementation.
- **Time Allocation:** Even though at first sight it looks like after the LLM is generating the code, there is just a little more work needed to make it work - the truth is that we think in this situation it is more like the Pareto principle (the 80-20): after we had 80% of the code - we were probably 20% way there. The work of making the functions do exactly what we wanted them to do, bug fixes and integrations with other parts of the code, took us the most of the time.

7 Reflection

Throughout the work on this project we learned a lot, had the chance to improve our software engineering skills, practice LLM usage and working in team. It was not easy for us especially because there are some tools we used in this project for the first time, but we think that this challenge was for the better.

In the end we succeeded doing two tasks instead of one and performed two demos - that made us very proud. In order to make the course better we suggest to create a task explaining page that will specify the requirement for each task to avoid confusion. In addition we suggest starting the integrations between different teams at an earlier stage of the course.

Another thing we think that will improve the course is implementing the use of Claude code in the course (the recent updates look very impressive).

At the bottom line we are very glad we took this course - we think that we learned important topics that will help us succeed in the software engineering with LLM world.